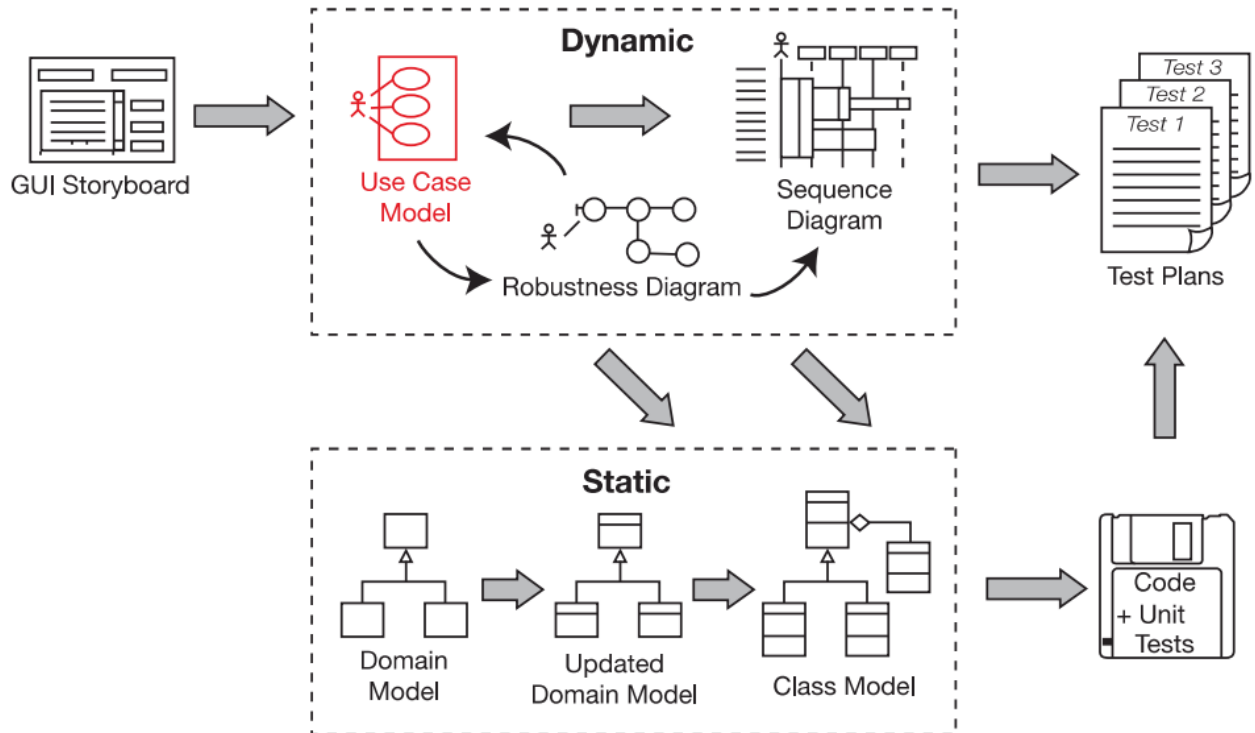


Bài 3: Mô hình hóa Use Case (Use Case Modeling)



3.1. Bối cảnh và vai trò của Use Case Modeling

Khi nhóm phát triển phần mềm đã xây dựng được Domain Model – tức mô hình miền khái niệm, mô tả cấu trúc tĩnh của hệ thống (các lớp miền, mối quan hệ has-a, is-a, thuộc tính và khái niệm cốt lõi) – thì câu hỏi tiếp theo xuất hiện một cách tự nhiên:

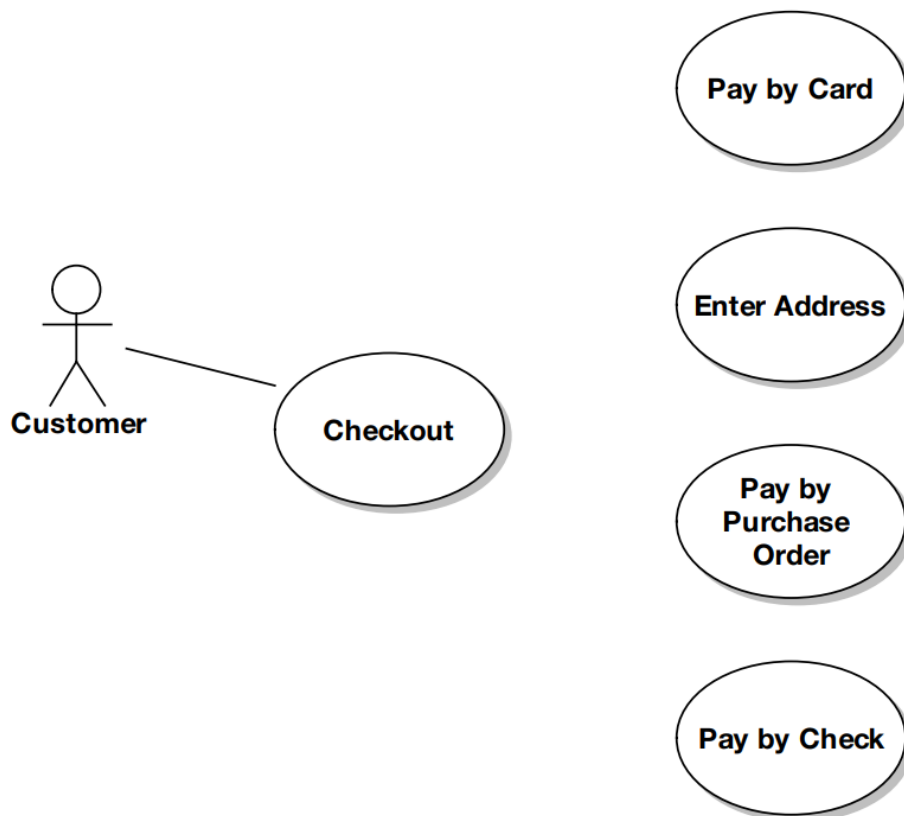
“Người dùng thật sự sẽ tương tác với những khái niệm này như thế nào trong các tình huống cụ thể?”

Domain Model giống như bản đồ tĩnh của thế giới thực mà hệ thống sẽ phản ánh, nhưng một bản đồ chỉ vẽ địa hình thì chưa thể hiện được các dòng chuyển động hay hành vi diễn ra trên đó. Phần mềm không chỉ “chứa đựng dữ liệu” mà còn phục vụ các hoạt động của con người. Vì vậy, để tiến gần hơn tới thiết kế và cài đặt, ta cần một công cụ có khả năng mô tả hành vi hệ thống từ góc nhìn người dùng – và đó chính là Use Case Modeling.

3.1.1 Cầu nối giữa mô hình tĩnh và hành vi động

Trong quy trình ICONIX, Use Case Modeling là bước bản lề, kết nối giữa Domain Modeling (Bài 2) và Robustness Analysis (Bài 4). Domain Model định nghĩa “các vật thể” của hệ thống, còn Use Case định nghĩa “các hành động” diễn ra với những vật thể đó. Hai phần này kết hợp với nhau để tạo thành nền tảng cho toàn bộ thiết kế và mã nguồn về sau.

Một cách trực quan, Domain Model giống như “danh sách nhân vật và bối cảnh”, còn Use Case chính là “kịch bản hành động” trong đó các nhân vật (actor) tương tác với hệ thống để đạt được một mục tiêu cụ thể. Nếu chỉ có domain model mà không có use case, ta có cấu trúc mà không có chuyển động; ngược lại, nếu chỉ có use case mà không có domain model, ta có các hành vi rời rạc không bám vào khái niệm rõ ràng.



3.1.2 Viết Use Case trong ngữ cảnh Domain Model

Một điểm then chốt trong phương pháp ICONIX là:

“Use cases are written in the context of the domain model, and assume that the initial domain model is wrong.”

Điều này có nghĩa là các use case không tồn tại độc lập. Khi viết kịch bản, nhóm phát triển luôn phải đặt chúng trong bối cảnh các khái niệm đã được xác định ở Domain Model — mỗi hành động, mỗi luồng dữ liệu trong use case đều nên liên hệ tới các lớp miền cụ thể. Nhưng đồng thời, ICONIX cũng khuyến khích một thái độ phản biện có hệ thống: hãy giả định rằng domain model ban đầu của bạn có thể thiếu, sai, hoặc chưa rõ.

Chính khi bạn mô tả các hành vi cụ thể, bạn sẽ phát hiện ra những lỗ hổng ngữ nghĩa hoặc khái niệm còn bỏ sót trong domain model: một mối quan hệ chưa được vẽ, một lớp còn thiếu, hay một trường hợp ngoại lệ chưa được xem xét. Việc viết use case

trở thành một công cụ kiểm thử khái niệm mạnh mẽ, giúp domain model phát triển theo hướng chính xác và thực dụng hơn.

3.2.3 Vai trò trung tâm trong toàn bộ ICONIX Process

Nếu Domain Modeling là “xương sống tĩnh”, thì Use Case Modeling chính là dòng máu vận hành toàn hệ thống. Mọi giai đoạn tiếp theo trong ICONIX đều dựa trên use case:

- Robustness Analysis sử dụng từng bước trong kịch bản use case để xác định các boundary–controller–entity và vẽ sơ đồ robustness.
- Sequence Diagrams được xây dựng trực tiếp từ kịch bản và kết quả robustness analysis, mô tả tương tác cụ thể giữa các đối tượng.
- Coding dựa trên các lớp và hành vi đã được xác định qua các bước trên.

Nếu use case không rõ ràng, thiếu luồng lỗi hoặc không bám domain model, toàn bộ thiết kế phía sau sẽ lung lay như xây nhà trên nền móng yếu. Ngược lại, một bộ use case rõ ràng, có logic, bao quát cả tình huống “sunny day” lẫn “rainy day” sẽ giúp nhóm phát triển tránh được hàng loạt lỗi thiết kế muộn, tiết kiệm chi phí và tăng tính nhất quán của toàn bộ hệ thống.

3.2. Từ yêu cầu ngôn ngữ tự nhiên đến Use Case

Các dự án phần mềm thường bắt đầu với những bản mô tả yêu cầu (requirements) viết bằng ngôn ngữ tự nhiên, đôi khi chỉ là email, ghi chú cuộc họp hoặc ý tưởng của khách hàng. Vấn đề là ngôn ngữ tự nhiên vốn mơ hồ, đa nghĩa và dễ bị diễn giải khác nhau. Đây là nguyên nhân gốc rễ của rất nhiều sai lệch trong quá trình phát triển phần mềm: nhóm kỹ thuật hiểu một kiểu, khách hàng nghĩ một kiểu, còn tester lại tưởng tượng ra một quy trình khác hoàn toàn.

Phương pháp ICONIX không cố gắng “chuẩn hóa” toàn bộ yêu cầu ngay lập tức — thay vào đó, nó sử dụng Use Case như một công cụ chuyển đổi có hệ thống từ ngôn ngữ tự nhiên sang mô hình hành vi có cấu trúc. Use Case giúp “ép” những mô tả mơ hồ trở thành những kịch bản tương tác cụ thể giữa người dùng và hệ thống, từ đó làm rõ từng hành vi, từng điều kiện, từng ngoại lệ.

Quá trình này thường bao gồm ba bước chính:

1. Đọc và hiểu yêu cầu để nhận diện các tác nhân (actor) và hành vi cốt lõi mà hệ thống cần hỗ trợ.
2. Xác định các Use Case sơ bộ dựa trên các động từ/hành động quan trọng trong mô tả yêu cầu.
3. Viết kịch bản (scenario) cho từng Use Case, bao gồm cả luồng chính (Basic Course) và các luồng thay thế (Alternate Courses).

Ví dụ, trong trường hợp hệ thống *Internet Bookstore*, yêu cầu có thể chỉ nói một cách đơn giản:

“Khách hàng có thể tìm kiếm sách, thêm vào giỏ hàng và thanh toán.”

Câu nói tưởng như đơn giản này ẩn chứa rất nhiều hành vi khác nhau: khách hàng là actor, các thao tác tìm kiếm — thêm — thanh toán là các hành động, và đằng sau mỗi hành động là một chuỗi tương tác cụ thể với hệ thống. Chính từ đây, nhóm phát triển sẽ lần lượt tách ra các Use Case như *Search Book*, *Add to Cart*, *Checkout*, và viết cho mỗi cái một kịch bản tương tác chi tiết.

3.2.1. Viết kịch bản như kể chuyện — không phải liệt kê chức năng

Một lỗi phổ biến khi sinh viên hoặc các nhóm non kinh nghiệm viết Use Case là biến nó thành danh sách chức năng khô khan hoặc mô tả GUI chi tiết. Ví dụ:

- “Người dùng nhập tên sách vào ô tìm kiếm.”
- “Người dùng nhấn nút màu xanh để gửi.”
- “Hệ thống trả về kết quả.”

Cách viết này không sai hoàn toàn, nhưng nó đánh mất bản chất của Use Case. ICONIX khuyến khích viết kịch bản như một câu chuyện ngắn (mini-story), mô tả dòng chảy hành động giữa actor và hệ thống theo cách tự nhiên, dễ đọc, nhưng có logic chặt chẽ.

Một kịch bản tốt không cần nhắc đến các chi tiết GUI, mà tập trung vào *ai làm gì* với hệ thống, và hệ thống phản hồi ra sao. Điều này không chỉ giúp kịch bản dễ hiểu mà còn cho phép tái sử dụng nó ở nhiều loại giao diện khác nhau (web, mobile, service API...) mà không cần viết lại toàn bộ.

3.2.2. Rainy-day scenarios – Đừng quên các tình huống “trời mưa”

Một trong những khuyến cáo quan trọng nhất của ICONIX khi viết Use Case là:

“Don’t forget the rainy-day scenarios.”

Rất nhiều nhóm chỉ viết luồng thành công (*sunny-day scenario*) — tức là luồng chính khi mọi thứ diễn ra thuận lợi — mà quên mô tả các tình huống ngoại lệ, lỗi, rẽ nhánh. Đây là một sai lầm lớn. Trong thực tế, chính những tình huống “trời mưa” này mới bộc lộ ra lỗ hổng của hệ thống và những thiếu sót trong Domain Model.

Rainy-day scenarios bao gồm nhiều loại:

- Người dùng chưa đăng nhập nhưng cố gắng thực hiện hành động cần quyền.
- Nhập dữ liệu không hợp lệ (quá dài, quá ngắn, sai định dạng).
- Thiếu dữ liệu cần thiết.
- Hệ thống hoặc dịch vụ bên ngoài không phản hồi.

- Các rẽ nhánh nghiệp vụ ít khi xảy ra nhưng vẫn phải được xử lý.

Ví dụ, trong Use Case “Write Book Review” của Internet Bookstore:

- Nếu người dùng chưa đăng nhập, hệ thống sẽ chuyển hướng đến trang đăng nhập trước khi cho phép viết review.
- Nếu review quá dài (>1MB), hệ thống từ chối và trả về thông báo lỗi.
- Nếu review quá ngắn (<10 ký tự), hệ thống cũng từ chối.

Những luồng này không chỉ quan trọng về mặt nghiệp vụ, mà còn giúp nhóm phát hiện ra các lớp, mối quan hệ và điều kiện kiểm tra còn thiếu trong Domain Model. Nói cách khác, việc viết rainy-day scenarios là một cơ chế “kiểm thử sớm” cho toàn bộ mô hình.

3.2.3. Gắn kịch bản với Domain Model để tránh lạc hướng

Một điểm đặc trưng của ICONIX là tất cả các Use Case phải được viết trong ngữ cảnh Domain Model. Điều này có nghĩa là: khi viết một bước trong kịch bản, ta luôn có thể chỉ ra nó liên quan tới lớp miền nào, hay tác động đến mối quan hệ nào trong Domain Model.

Ví dụ:

- “Customer thêm Book vào Shopping Cart” → liên quan đến các lớp *CustomerAccount*, *Book*, *ShoppingCart*, và mối quan hệ *ShoppingCart has-a LineItem*.
- “System kiểm tra giới hạn độ dài của Review” → liên quan đến lớp *BookReview*, thuộc tính *ReviewText*, và quy tắc nghiệp vụ.

Việc này có hai lợi ích quan trọng:

1. Giúp nhóm phát hiện kịp thời những khái niệm chưa có trong Domain Model.
2. Ngăn kịch bản trôi đi theo hướng mô tả GUI hoặc hành vi không có cơ sở khái niệm rõ ràng.

Chính vì vậy, ICONIX nhấn mạnh: viết Use Case không phải là một hoạt động độc lập, mà là một vòng lặp liên tục với Domain Modeling. Mỗi khi kịch bản phát hiện ra khái niệm mới hoặc lỗi mô hình, Domain Model cần được cập nhật tương ứng.

3.2.4. Kiểm duyệt Use Case sớm để tránh trả giá muộn

Một Use Case được viết rõ ràng, có đầy đủ rainy-day scenarios, gắn chặt với Domain Model, sẽ trở thành nền tảng đáng tin cậy cho các bước tiếp theo: Robustness Analysis, Sequence Diagram và cuối cùng là lập trình. Ngược lại, nếu Use Case mơ hồ, thiếu kịch bản lỗi, hoặc không logic, thì các bước sau sẽ phải “chữa cháy” liên tục — dẫn tới hệ thống rối rắm, khó bảo trì và nhiều lỗi nghiệp vụ.

ICONIX do đó xem việc review Use Case là một checkpoint chính thức trong quy trình phát triển. Đây là thời điểm lý tưởng để tất cả các bên (phân tích, thiết kế, lập trình,

khách hàng) ngồi lại, đọc từng kịch bản, kiểm tra logic, thảo luận các trường hợp biên và xác nhận tính đúng đắn. Mọi lỗi phát hiện ở giai đoạn này đều rẻ hơn rất nhiều so với khi đã code.

3.3. Top 10 Use Case Modeling Guidelines

Viết Use Case tưởng chừng đơn giản — chỉ cần mô tả các hành vi mà người dùng có thể thực hiện với hệ thống. Nhưng trên thực tế, rất nhiều nhóm sinh viên hoặc kỹ sư phần mềm mắc lỗi ở chính giai đoạn này, khiến toàn bộ phần thiết kế phía sau trở nên rối rắm, không thể mở rộng, hoặc tệ hơn là sai hướng ngay từ đầu. ICONIX đề xuất một tập hợp 10 hướng dẫn kinh điển giúp tránh những sai lầm đó. Đây không chỉ là “mẹo viết tài liệu” mà là những quy tắc tư duy nền tảng khi mô hình hóa Use Case.

3.1. Guideline 10 — Viết Use Case theo ngôn ngữ tự nhiên, không phải thuật ngữ kỹ thuật

Điểm xuất phát của Use Case là yêu cầu nghiệp vụ (business requirements), vì vậy kịch bản phải được viết bằng **ngôn ngữ mà cả khách hàng lẫn đội ngũ kỹ thuật đều hiểu được**. Nếu ngay từ đầu ta đưa vào các thuật ngữ kỹ thuật như API, class, field, bảng dữ liệu... thì kịch bản sẽ nhanh chóng trở thành tài liệu dành riêng cho lập trình viên và mất tác dụng như một công cụ giao tiếp chung.

ICONIX nhấn mạnh rằng Use Case phải đọc như một đoạn mô tả tương tác trong đời thực. Ví dụ:

“Customer clicks the ‘Write Review’ button, the system shows the Write Review screen...” nghe rất tự nhiên và rõ ràng. Trong khi đó, nếu viết kiểu: “OnClick event is triggered and Controller invokes the ReviewService.create() method...” thì đây đã là Sequence Diagram hoặc chi tiết thiết kế, không còn là Use Case nữa.

Một kịch bản Use Case tốt là tài liệu mà khách hàng đọc được, hiểu được và xác nhận được ngay lập tức.

3.2. Guideline 9 — Viết trong ngữ cảnh Domain Model, tránh mô tả GUI

Một lỗi rất thường gặp là biến Use Case thành bản mô tả chi tiết của giao diện người dùng. Điều này có hai tác hại lớn:

1. Nó gắn chặt kịch bản với một kiểu giao diện cụ thể, khiến hệ thống khó thay đổi giao diện trong tương lai.
2. Nó làm lu mờ bản chất nghiệp vụ, khiến nhóm không phát hiện được các lớp miền (domain classes) và mối quan hệ quan trọng.

ICONIX yêu cầu mọi Use Case phải được viết dựa trên các khái niệm của Domain Model. Khi viết mỗi bước, người phân tích nên tự hỏi: “Bước này đang tương tác với lớp miền nào? Mối quan hệ gì được sử dụng ở đây?” Việc này tạo nên sự liên kết chặt chẽ giữa phần tĩnh (domain) và phần động (use case) của hệ thống, giúp mô hình hóa trở nên nhất quán và có thể truy vết được.

3.3. Guideline 8 — Giữ Use Case ở mức khái niệm, tránh chi tiết kỹ thuật

Tương tự Domain Model, Use Case không phải là nơi để bàn về chi tiết lập trình hay kiến trúc. Ví dụ, đừng nói “Hệ thống lưu Book Review vào bảng review_tbl bằng câu lệnh SQL INSERT” — đó không phải Use Case, đó là chi tiết triển khai.

Use Case chỉ cần mô tả rằng “System stores the Book Review so that it can be moderated later.” Việc lưu bằng SQL, NoSQL hay file hệ thống sẽ do các giai đoạn thiết kế và cài đặt quyết định.

Giữ Use Case ở mức khái niệm giúp tài liệu linh hoạt, bền vững và dễ cập nhật hơn khi công nghệ thay đổi.

3.4. Guideline 7 — Xác định rõ Actor chính và Actor phụ

Mỗi Use Case thường có **một actor chính (primary actor)** — người hoặc hệ thống khởi xướng hành động — và có thể có các actor phụ (secondary actors), đóng vai trò hỗ trợ hoặc phản hồi. Việc xác định rõ vai trò này giúp kịch bản mạch lạc hơn và tránh nhầm lẫn khi phân tích tương tác. Ví dụ:

- Trong Use Case “Write Book Review”, actor chính là *Customer*, actor phụ có thể là *Moderator* (người kiểm duyệt), người sẽ tiếp nhận review sau khi gửi.
- Trong Use Case “Checkout”, actor chính là *Customer*, còn *Payment Gateway* là actor phụ.

Nếu không xác định rõ vai trò ngay từ đầu, kịch bản có thể trở nên lộn xộn, hoặc tệ hơn là thiết kế hệ thống nhằm hướng về sau.

3.5. Guideline 6 — Bắt đầu từ luồng chính (Basic Course), sau đó mới thêm luồng phụ (Alternate Courses)

Khi viết Use Case, hãy luôn bắt đầu bằng luồng chính — *Basic Course*. Đây là kịch bản thành công điển hình, mô tả quá trình actor và hệ thống tương tác thuận lợi nhất.

Sau khi luồng chính hoàn thiện, ta mới thêm các luồng phụ (*Alternate Courses*) để xử lý ngoại lệ, rẽ nhánh, hoặc các điều kiện đặc biệt.

Việc này giúp kịch bản không bị phân nhánh lung tung từ đầu, đồng thời đảm bảo rằng mọi alternate flow đều có một mốc tham chiếu rõ ràng trong luồng chính. Đây là cách ICONIX kiểm soát tính nhất quán và dễ đọc của Use Case.

3.6. Guideline 5 — Don’t forget the rainy-day scenarios

Đây là nguyên tắc đặc biệt quan trọng, đến mức trong sách ICONIX, tác giả nhấn mạnh nó nhiều lần như một “cạm bẫy chết người” nếu bỏ qua. Rainy-day scenarios chính là nơi hệ thống bị thử thách thực sự. Nếu chỉ viết sunny-day scenario, Use Case có thể trông đẹp, nhưng nó không giúp phát hiện lỗi nghiệp vụ, không kiểm chứng Domain Model, và dẫn đến những bất ngờ khó chịu khi code đã viết xong. Hãy tưởng tượng một hệ thống đặt hàng mà Use Case “Place Order” không mô tả chuyện khách

nhập thẻ sai, hàng hết, hoặc kết nối thanh toán lỗi. Đến khi chạy thật, toàn bộ hệ thống có thể sụp đổ chỉ vì những kịch bản “trời mưa” bị lãng quên.

3.7. Guideline 4 — Mỗi Use Case nên có độ dài hợp lý

Một Use Case quá ngắn (chỉ vài dòng) thường là dấu hiệu của việc mô tả chức năng ở mức quá thấp, hoặc chỉ đơn giản là ghi chú. Ngược lại, một Use Case quá dài (trên 3 trang) có thể là dấu hiệu rằng bạn đang trộn nhiều kịch bản khác nhau vào một. Một Use Case tốt nên có độ dài vừa phải, đủ để mô tả luồng chính và các luồng phụ một cách mạch lạc mà không bị loãng. Thông thường, luồng chính khoảng 5–10 bước, mỗi alternate flow 2–4 bước là hợp lý.

3.8. Guideline 3 — Đừng mô tả Use Case bằng biểu đồ lớp hay sequence diagram

Đây là lỗi phổ biến của những nhóm “đi tắt” từ Domain Model sang thiết kế mà không viết kịch bản rõ ràng. Khi bỏ qua bước Use Case, tài liệu dễ trở thành một tập hợp các sơ đồ kỹ thuật rời rạc, không có ngữ cảnh hành vi. Use Case đóng vai trò cầu nối hành vi. Nếu bạn thay nó bằng sơ đồ, bạn đang bỏ qua bước chuyển quan trọng nhất từ yêu cầu sang thiết kế.

3.9. Guideline 2 — Luôn review Use Case như một checkpoint chính thức

Use Case không phải tài liệu viết xong là xong. Nó phải được xem xét nghiêm túc bởi toàn bộ nhóm — phân tích, thiết kế, code, test, và cả khách hàng. ICONIX coi việc review Use Case là một trong bốn *milestone* kiểm duyệt chính của quy trình. Đây là cơ hội vàng để bắt lỗi logic sớm, trước khi bước sang Robustness Analysis. Một Use Case tốt sẽ tiết kiệm cho bạn hàng chục giờ debug sau này.

3.10. Guideline 1 — Đừng mong Use Case hoàn hảo ngay từ đầu

Một Use Case sẽ không thể hoàn thiện trong lần viết đầu tiên. Nó là kết quả của quá trình lặp đi lặp lại giữa việc đọc yêu cầu, xây dựng Domain Model, brainstorm nhóm, và sửa lỗi sau review. ICONIX khuyến khích cách tiếp cận “**assume that the initial domain model is wrong**” — tức là giả định rằng mô hình và Use Case đầu tiên chắc chắn có lỗi, để sẵn sàng tinh chỉnh liên tục thay vì bảo thủ. Chính nhờ vòng lặp này mà Use Case và Domain Model dần trở nên chính xác, phản ánh đúng không gian vấn đề.

Storyboards, Active Voice và GUI Prototype

Bên cạnh các nguyên tắc viết use case, ICONIX cũng nhấn mạnh việc sử dụng storyboards (bản phác giao diện), nguyên mẫu giao diện người dùng (GUI prototypes), và thể chủ động (active voice) như các công cụ hỗ trợ tư duy hành vi.

Thể chủ động: Việc viết ở thể chủ động giúp câu văn rõ ràng, tập trung vào “ai làm gì với hệ thống”, thay vì những câu bị động mơ hồ như “Khả năng đăng nhập được cung cấp cho người dùng...”. Câu chủ động kiểu “Người dùng nhập tên đăng nhập và mật

khẩu, hệ thống kiểm tra thông tin và đăng nhập người dùng” giúp phát hiện hành vi cần thiết dễ dàng hơn.

Internet Bookstore - Edit Shopping Cart		
Items in Your Shopping Cart	Price:	Qty:
Domain Driven Design	\$42.65	1
Extreme Programming Refactored	\$29.65	1
		Update

Storyboards & GUI Prototypes: Storyboard là các bản phác tay hoặc sơ đồ giao diện dạng “Napkin Look & Feel” được dùng song song khi viết use case. Chúng giúp tất cả các bên hình dung rõ hành vi và giao diện mà không bị sa đà vào chi tiết kỹ thuật. ICONIX khuyến nghị sử dụng các bản vẽ đường nét đơn giản, đính kèm vào use case để minh họa luồng sự kiện/phản hồi một cách trực quan. Việc sử dụng storyboard đúng cách giúp tăng tốc quá trình viết kịch bản, tạo ngôn ngữ chung giữa nhóm kỹ thuật và người dùng, đồng thời hỗ trợ mạnh mẽ cho bước thiết kế sau này.

Viết Use Case theo Luồng Sự kiện/Phản hồi

Một use case hiệu quả nên được viết như một cuộc đối thoại hai chiều giữa người dùng và hệ thống, thay vì chỉ là một danh sách hành động một phía. ICONIX gọi đây là phương pháp viết theo luồng sự kiện/phản hồi (flow of events/responses)

Trong thực tế, một use case thường bắt đầu bằng một hành động từ phía người dùng (ví dụ: nhấn nút, gửi biểu mẫu), hoặc từ hệ thống (ví dụ: hiển thị màn hình, gửi thông báo). Bên còn lại sẽ phản hồi tương ứng. Chuỗi này tiếp tục xen kẽ cho đến khi hoàn thành kịch bản.

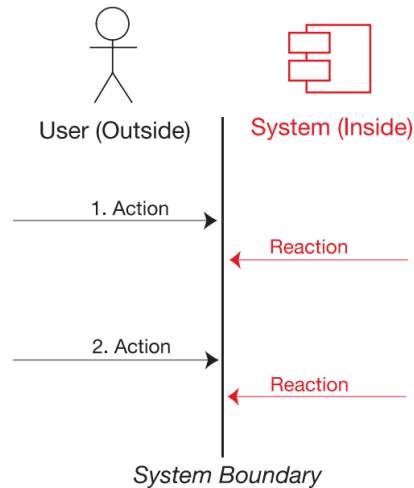
Ví dụ, thay vì viết mơ hồ:

“Hệ thống cho phép người dùng đăng nhập thông qua cơ chế bảo mật.”

Hãy viết theo kiểu sự kiện/phản hồi:

“Hệ thống hiển thị màn hình Đăng nhập. Người dùng nhập tên đăng nhập và mật khẩu, sau đó nhấn nút Đăng nhập. Hệ thống tìm kiếm hồ sơ người dùng bằng tên đăng nhập và kiểm tra mật khẩu. Sau đó, hệ thống đăng nhập người dùng.”

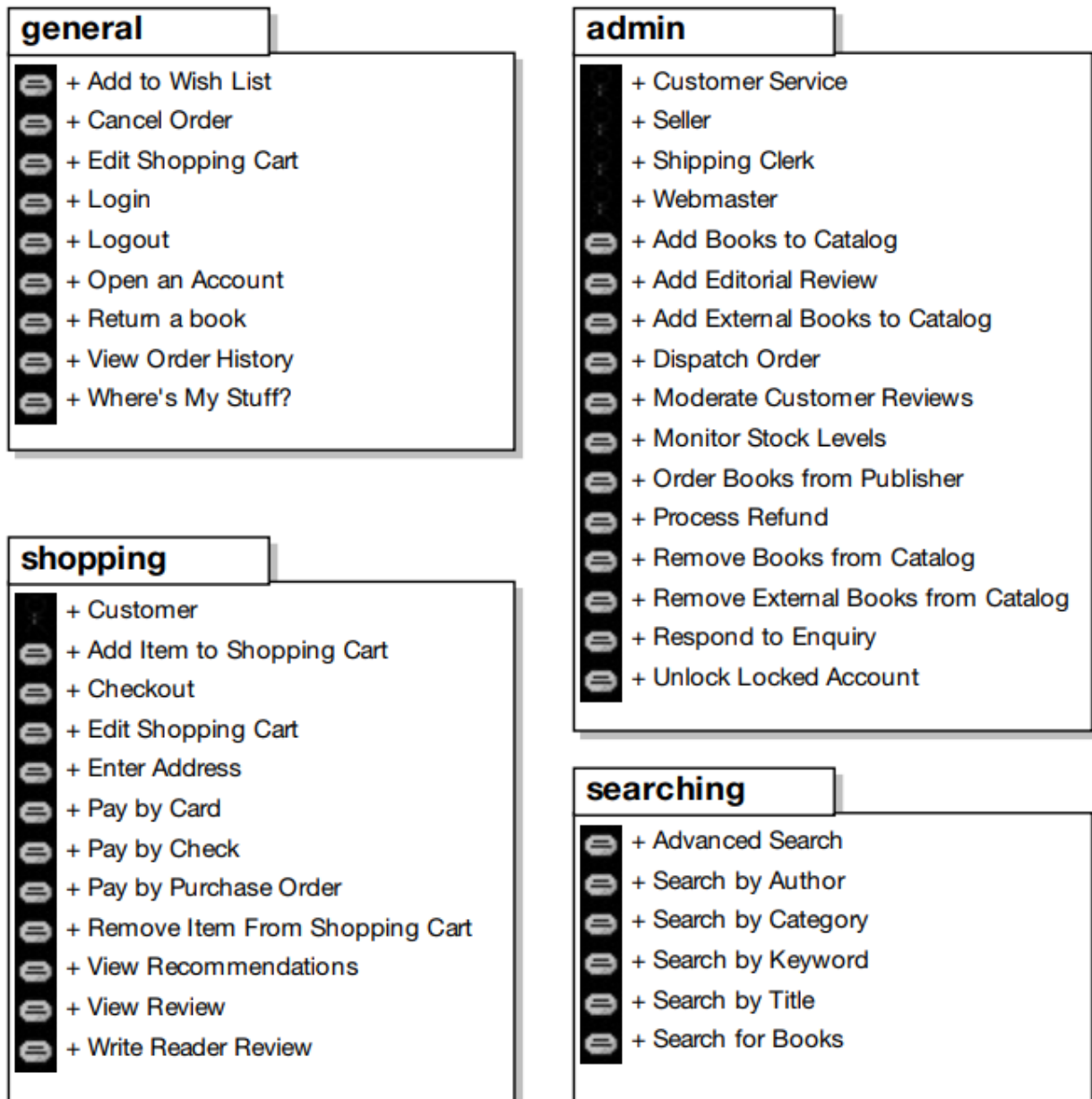
ICONIX xem đây là cách tiếp cận “từ ngoài vào trong” — mô tả rõ tương tác thực tế trước khi chuyển sang mô hình kỹ thuật. Đây cũng chính là cầu nối quan trọng giữa Use Case và Robustness Analysis ở bước tiếp theo.



3.4. Organizing Use Cases into Packages

Khi số lượng use cases trong một hệ thống bắt đầu tăng lên, việc quản lý chúng trở nên phức tạp một cách đáng ngạc nhiên. Trong những dự án nhỏ, có thể nhóm chỉ có khoảng 5–10 use cases; nhưng ở những hệ thống thực tế như Internet Bookstore, con số này dễ dàng vượt quá 30–40 use cases chỉ sau một vài vòng phân tích. Nếu không có một cấu trúc tổ chức rõ ràng, tài liệu nhanh chóng trở thành một “mớ hỗn độn” — người đọc không biết bắt đầu từ đâu, các use cases bị trùng lặp về nội dung, hoặc tệ hơn là mâu thuẫn ngầm mà không ai nhận ra.

Đây chính là lúc khái niệm Use Case Package phát huy tác dụng. Việc nhóm các use cases vào các package không chỉ giúp quản lý tốt hơn, mà còn giúp bộc lộ cấu trúc chức năng tự nhiên của hệ thống, giống như cách một thư viện có hệ thống phân loại sách để bạn có thể nhanh chóng tìm được quyển cần đọc. ICONIX không xem đây là bước hình thức, mà là một công cụ phân tích thực sự.



3.4.1. Vì sao cần tổ chức Use Case thành Packages

Một hệ thống phần mềm thường bao gồm nhiều phân hệ chức năng (functional subsystems), mỗi phân hệ giải quyết một nhóm yêu cầu nghiệp vụ tương đối độc lập. Ví dụ, Internet Bookstore có thể chia thành ba nhóm lớn: nhóm chức năng dành cho khách hàng (Customer Functions), nhóm chức năng xử lý đơn hàng và thanh toán (Order and Payment Functions), và nhóm chức năng quản trị hệ thống (Administration Functions).

Nếu không phân chia rõ ràng, các use cases của ba nhóm này sẽ bị trộn lẫn vào nhau, khiến việc đọc hiểu, bảo trì, hay phân công nhiệm vụ trở nên rất khó khăn. Việc tổ chức use cases vào các package tương ứng giúp làm rõ ranh giới chức năng và trách nhiệm nghiệp vụ. Điều này cũng hỗ trợ mạnh mẽ cho bước thiết kế kiến trúc sau này,

khi ta có thể ánh xạ các package use case sang các module hoặc component tương ứng trong hệ thống.

3.4.2. Cách tiếp cận: Từ Domain Model sang Use Case Package

ICONIX nhấn mạnh rằng việc phân nhóm use case không nên bắt đầu từ cấu trúc giao diện (GUI), mà từ Domain Model. Domain Model đóng vai trò như bản đồ khái niệm tính của hệ thống, cho ta thấy các nhóm thực thể và mối quan hệ tự nhiên giữa chúng. Chính từ đây, ta có thể xác định những “cụm chức năng tự nhiên” và tổ chức các use cases tương ứng.

Ví dụ, trong Internet Bookstore, nhóm các lớp như Book, BookCatalog, SearchMethod, BookReview tạo thành một cụm khái niệm xoay quanh việc tìm kiếm và xem sách. Điều này gợi ý một package use case như “Search and Browse”. Tương tự, cụm ShoppingCart, Order, PaymentType, CustomerAccount gợi ý một package “Ordering and Checkout”. Các chức năng liên quan đến quản lý đối tác, tài khoản tổng, hoặc fulfillment sẽ thuộc về một package quản trị. Điều quan trọng là: domain model định hướng cách nhóm use case, chứ không phải giao diện người dùng hay code structure. Điều này giúp các package phản ánh cấu trúc nghiệp vụ thật sự của hệ thống.

3.4.3. Đảm bảo package phản ánh ranh giới chức năng thực sự

Một lỗi phổ biến của sinh viên là nhóm use case dựa trên menu hoặc màn hình GUI. Ví dụ, họ sẽ tạo package “Main Menu”, “Account Menu”, “Admin Menu”... Nghe thì hợp lý, nhưng thực ra đây chỉ là phản ánh cấu trúc giao diện, không phải cấu trúc nghiệp vụ. Khi giao diện thay đổi (ví dụ từ web sang mobile app), cách nhóm này lập tức trở nên lỗi thời.

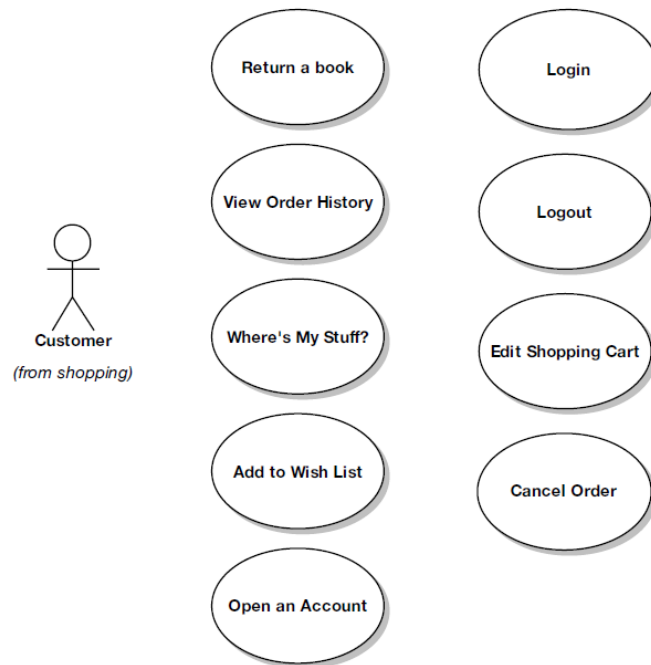
ICONIX yêu cầu mỗi package phải tương ứng với một nhóm chức năng nghiệp vụ (functional responsibility) trong hệ thống. Việc này không chỉ giúp tài liệu bền vững trước thay đổi giao diện, mà còn giúp kiến trúc phần mềm dễ dàng ánh xạ sang các module có trách nhiệm rõ ràng. Ví dụ, package “Ordering and Checkout” có thể trở thành một module backend xử lý tất cả logic liên quan đến đặt hàng, bất kể giao diện người dùng thay đổi ra sao.

3.4.4. Từ Use Case Package đến Use Case Diagram

Sau khi xác định được các package, bước kế tiếp là thể hiện chúng trên Use Case Diagram. Mỗi package sẽ trở thành một “vùng chứa” (container) gồm các use case có liên quan. Các actor tương tác với các use case trong package thông qua các đường kết nối.

Ví dụ, package “Customer Functions” sẽ chứa các use cases như “Search Book”, “Add to Cart”, “Write Review”, “Login/Logout”. Actor Customer sẽ tương tác trực tiếp với hầu hết các use case này. Trong khi đó, package “Administration” có thể bao gồm “Moderate Review”, “Manage Catalog”, với actor là Moderator hoặc Admin.

Sơ đồ này giúp người đọc có cái nhìn “từ trên xuống” (top–down) về toàn bộ hệ thống: các nhóm chức năng chính là gì, actor nào tương tác với nhóm nào, và các chức năng chi tiết nằm ở đâu. Đây là công cụ cực kỳ hiệu quả trong các buổi review kiến trúc hoặc khi giới thiệu hệ thống cho thành viên mới.



3.4.5. Hỗ trợ phân công và lập kế hoạch dự án

Một lợi ích thực tế nữa của việc tổ chức use cases vào packages là hỗ trợ việc phân công công việc và lập kế hoạch phát triển. Trong một nhóm lớn, các package thường được giao cho các nhóm con hoặc cá nhân phụ trách. Mỗi nhóm sẽ chịu trách nhiệm phân tích chi tiết, viết kịch bản, thiết kế và triển khai cho toàn bộ use cases trong package được giao.

Ví dụ, một nhóm có thể đảm nhận toàn bộ package “Search and Browse”, bao gồm các use cases liên quan đến tra cứu và hiển thị thông tin sách; trong khi nhóm khác tập trung vào package “Ordering and Checkout”. Cách chia này tương ứng khá tự nhiên với việc phân chia module backend hoặc microservice trong các hệ thống hiện đại. Nhờ vậy, quy trình phát triển có thể được song song hóa, giảm phụ thuộc chéo, đồng thời vẫn đảm bảo tính thống nhất vì mọi nhóm đều dựa trên một domain model chung.

3.4.6. Package giúp kiểm soát sự mở rộng của hệ thống

Khi hệ thống mở rộng theo thời gian, số lượng use cases tăng lên là điều không thể tránh khỏi. Tuy nhiên, nếu các use case đã được tổ chức tốt vào packages ngay từ đầu, việc thêm mới hoặc chỉnh sửa trở nên dễ dàng hơn rất nhiều. Ta có thể nhanh chóng xác định package liên quan, cập nhật hoặc bổ sung mà không phải rà soát toàn bộ tài liệu. Hơn nữa, cấu trúc package cũng giúp nhận diện những khu vực đang phát triển quá phức tạp. Nếu một package trở nên quá lớn, đó là dấu hiệu cho thấy ta cần

chia nhỏ nó thành các package con hoặc tái cấu trúc lại domain model. Điều này tạo ra một cơ chế phản hồi tự nhiên, giúp giữ cho tài liệu luôn trong tình trạng “gọn gàng có tổ chức”.

3.5. Viết kịch bản use case

Một ví dụ điển hình giúp minh họa cách viết Use Case theo quy trình ICONIX là Write Reader Review, trong bối cảnh hệ thống Internet Bookstore. Đây là trường hợp cho thấy rõ cách người dùng tương tác với hệ thống khi viết đánh giá cho một cuốn sách, bao gồm cả luồng hành vi chính (basic course) và các luồng thay thế (alternate courses) tương ứng với các tình huống ngoại lệ. Mục tiêu của use case này là mô tả mạch lạc chuỗi hành động qua lại giữa Customer và System, đồng thời chỉ ra cách hệ thống kiểm tra dữ liệu và phản hồi trong các điều kiện khác nhau.

BASIC COURSE:

The Customer clicks the Write Review button for the book currently being viewed, and the system shows the Write Review screen. The Customer types in a Book Review, gives it a Book Rating out of five stars, and clicks the Send button. The system ensures that the Book Review isn't too long or short, and that the Book Rating is within one and five stars. The system then displays a confirmation screen, and the review is sent to a Moderator, ready to be added.

ALTERNATE COURSES:

User not logged in: The user is first taken to the Login screen and then to the Write Review screen once he is logged in.

The user enters a review that is too long (text > 1MB): The system rejects the review and responds with a message explaining why the review was rejected.

The review is too short (< 10 characters): The system rejects the review.

Trong **Basic Course**, Customer đang xem thông tin chi tiết của một cuốn sách và nhấn nút *Write Review*. Hệ thống hiển thị màn hình *Write Review screen* cho phép người dùng nhập đánh giá. Customer sau đó gõ nội dung nhận xét (Book Review), chọn mức đánh giá (Book Rating) theo thang năm sao và nhấn nút *Send*. Hệ thống tiến hành kiểm tra để bảo đảm rằng nội dung nhận xét không quá ngắn hoặc quá dài, đồng thời giá trị đánh giá nằm trong khoảng từ một đến năm sao. Nếu dữ liệu hợp lệ, hệ thống hiển thị màn hình xác nhận (confirmation screen) và gửi bản nhận xét đến Moderator để chờ phê duyệt trước khi được thêm vào hệ thống. Đây là luồng hành vi “trời nắng” (sunny-day scenario), mô tả cách thức tương tác điển hình khi người dùng đã đăng nhập và nhập dữ liệu đúng quy cách, còn hệ thống vận hành bình thường.

Bên cạnh luồng chính, ICONIX Process đặc biệt nhấn mạnh tầm quan trọng của **Alternate Courses**, hay còn gọi là các tình huống “trời mưa” (rainy-day scenarios), nhằm bao quát những trường hợp phát sinh lỗi hoặc điều kiện bất thường. Trong use case này, có ba tình huống thay thế cơ bản. Thứ nhất, nếu người dùng chưa đăng

nhập, khi nhấn *Write Review*, hệ thống sẽ chuyển hướng đến màn hình đăng nhập (Login screen); sau khi quá trình đăng nhập hoàn tất, người dùng sẽ được đưa trở lại màn hình viết đánh giá. Thứ hai, nếu phần nội dung nhận xét dài quá giới hạn cho phép (ví dụ trên 1 MB), hệ thống sẽ từ chối, kèm theo thông báo giải thích lý do. Thứ ba, nếu nội dung đánh giá quá ngắn (ít hơn 10 ký tự), hệ thống cũng từ chối và thông báo lỗi cho người dùng. Những tình huống này không chỉ đảm bảo rằng use case phản ánh đúng hành vi thực tế của người dùng mà còn đóng vai trò như một cơ chế kiểm thử khái niệm sớm cho Domain Model, giúp phát hiện kịp thời các điều kiện ràng buộc hoặc quy tắc nghiệp vụ còn thiếu, chẳng hạn như kiểm soát độ dài chuỗi văn bản hoặc xử lý trạng thái đăng nhập.

Kết luận

Mô hình hóa Use Case giữ vai trò trung tâm trong quy trình ICONIX, kết nối giữa mô hình miền tĩnh (Domain Model) và phân tích hành vi, đảm bảo rằng hệ thống phần mềm vừa phản ánh đúng cấu trúc nghiệp vụ, vừa đáp ứng được các tình huống tương tác thực tế của người dùng. Thông qua các kịch bản Use Case, nhóm phát triển chuyển hóa yêu cầu mơ hồ bằng ngôn ngữ tự nhiên thành những luồng hành động rõ ràng, có thể kiểm chứng và dễ dàng chuyển tiếp sang các bước thiết kế kỹ thuật. Một bộ Use Case được viết tốt không chỉ bao quát đầy đủ luồng hành vi chính (sunny-day) mà còn xử lý cẩn trọng các tình huống ngoại lệ (rainy-day), được viết trong ngữ cảnh của Domain Model và tuân thủ các hướng dẫn chuẩn mực. Đây là nền tảng để xây dựng Robustness Diagram, Sequence Diagram, và cuối cùng là cài đặt hệ thống một cách chính xác, giảm thiểu rủi ro và chi phí sửa lỗi về sau. Ngoài ra, việc tổ chức các Use Case thành Packages phản ánh ranh giới chức năng thực sự giúp quản lý tài liệu tốt hơn, hỗ trợ phân công công việc và tạo điều kiện cho mở rộng hệ thống về sau. Tất cả những yếu tố này kết hợp lại làm cho Use Case Modeling trở thành một “điểm bản lề” chiến lược trong toàn bộ quy trình phát triển phần mềm — nơi mà yêu cầu, hành vi người dùng, cấu trúc nghiệp vụ và thiết kế kỹ thuật gặp nhau một cách có hệ thống.

Câu hỏi ôn tập

1. Giải thích vai trò của Use Case Modeling trong quy trình ICONIX. Vì sao nó được xem là “cầu nối” giữa Domain Model và Robustness Analysis?
2. Nêu lý do tại sao Use Case phải được viết trong ngữ cảnh của Domain Model. Điều gì sẽ xảy ra nếu bỏ qua nguyên tắc này?
3. Phân biệt giữa **sunny-day scenario** và **rainy-day scenario**. Tại sao rainy-day scenario đặc biệt quan trọng trong thiết kế hệ thống?
4. Trình bày ba bước chính để chuyển từ yêu cầu ngôn ngữ tự nhiên sang Use Case theo ICONIX.
5. Tại sao ICONIX khuyến khích viết Use Case như một câu chuyện (storytelling) thay vì liệt kê chức năng?
6. Nêu và giải thích ngắn gọn ba trong số **Top 10 Use Case Modeling Guidelines** mà bạn cho là quan trọng nhất.

7. Phân tích ý nghĩa của nguyên tắc “assume that the initial domain model is wrong” trong quá trình viết Use Case.
8. Vì sao việc tổ chức Use Case thành Packages lại cần bắt đầu từ Domain Model chứ không phải từ cấu trúc giao diện (GUI)?
9. Cho ví dụ một lỗi phổ biến khi viết Use Case và cách khắc phục dựa theo hướng dẫn của ICONIX.
10. Trong ví dụ “Write Book Review” của Internet Bookstore, hãy mô tả lại **Basic Course** và nêu ít nhất hai **Alternate Courses** tương ứng với các tình huống “trời mưa”.